

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Balan Gabriel FAF-241

Report

Laboratory Work No. 7

Operating Systems

Topic: Three Small Programs: Processes, Redirection, a Pipe

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Purpose of the Work 3

2 Part 1 – `fork` + `exec` + `wait` 3

2.1 Observations 4

3 Part 2 – A Zombie, on Purpose 5

3.1 Observations 5

4 Part 3 – Redirection: `ls -l > out.txt` 6

4.1 Observations 7

5 Part 4 – A Pipe: `ls /etc | wc -l` 8

5.1 Observations 9

6 Conclusion 10

1. Purpose of the Work

The purpose of this laboratory work is to see directly how the shell creates processes and connects their input and output. I used small C programs to experiment with `fork`, `exec` and `wait`, intentionally created a zombie process, reproduced output redirection by manipulating file descriptors, and finally built the equivalent of `ls /etc | wc -l` manually using a pipe and two child processes.

2. Part 1 – `fork` + `exec` + `wait`

The first program reproduces the basic steps a shell uses when it runs an external command: the process forks, the child replaces itself using `exec`, and the parent waits for the child to finish.

run.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command: ls -l */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status)
    );
    return 0;
}
```

With `ls -l`, my output was:

Output – `ls -l`

```
parent: my PID is 8213
child: my PID is 8214, my parent is 8213
total 24
-rwxrwxr-x 1 findredhundred findredhundred 16392 Oct 19 13:43 run
-rw-rw-r-- 1 findredhundred findredhundred   911 Oct 19 13:43 run.c
parent: child 8214 ended with status 0
```



Figure 1.1 – Running the program with `ls -l`.

I then changed only the `execlp` call so that the child became `date`.

Output – `date`

```
parent: my PID is 10370
child: my PID is 10371, my parent is 10370
Mon Oct 19 01:48:58 PM UTC 2026
parent: child 10371 ended with status 0
```

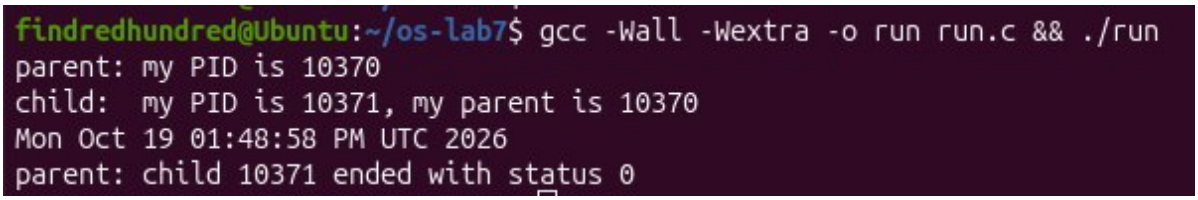


Figure 1.2 – Replacing the child process with `date`.

Finally, I tried to execute a program which does not exist.

Output – missing program

```
parent: my PID is 10938
child: my PID is 10940, my parent is 10938
exec: No such file or directory
parent: child 10940 ended with status 127
```

2.1. Observations

1. Why is `perror("exec")` printed only with `nosuchprog`?

```
findredhundred@Ubuntu:~/os-lab7$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 10938
child:  my PID is 10940, my parent is 10938
exec: No such file or directory
parent: child 10940 ended with status 127
```

Figure 1.3 – Failed `exec` and exit status 127.

Exec replaces the code with that of the file we provide. If the file successfully loads, the error line is replaced, hence it can never execute. It only executes in the case where the file couldn't be loaded.

2. Which number does the child print as "my parent", and whose PID is it?

The child prints the process id of the parent from which it has forked. For example, in my first run the child printed parent PID 8213, which is exactly the PID printed by the original process before the fork.

3. What status does the parent see with `ls`, and what status with `nosuchprog`?

For `ls` the child process returns with value 0, signaling success. With `nosuchprog` it returns with 127, meaning command not found.

3. Part 2 – A Zombie, on Purpose

For this part I used the same process structure, but placed `sleep(20)` in the parent before `waitpid`. This intentionally leaves an interval where the child has already finished but the parent has not collected its exit status yet.

Relevant change in zombie.c

```
int status;
sleep(20);
waitpid(pid, &status, 0);
```

I compiled it, started it in the background, and inspected the process table while the parent was sleeping.

Commands

```
gcc -Wall -Wextra -o zombie zombie.c
./zombie > /dev/null &
sleep 2; ps -o pid,ppid,stat,cmd
```

3.1. Observations

1. Which letter marks the zombie process?

Obviously it has `Z` in the STAT column, meaning we have a zombie process. In my screenshot the defunct child is shown as `[zombie] <defunct>` because that particular run also shows



Figure 2.1 – A child process in the zombie state while the parent has not called `waitpid` yet.

`exec: No such file or directory`, but the important part is the `Z` state.

2. Why does the zombie exist for about 20 seconds, and what makes it disappear?

The `sleep(20)` line in the code causes the process to be a zombie for that interval. A zombie is defined as a process that has returned but waits for the parent to collect the return value. This is done by the `wait` / `waitpid` command. However we wait for 20 seconds before calling `wait`, meaning this 20 second interval is when the child is a zombie. After the sleep ends, `waitpid` collects it and the defunct entry disappears.

4. Part 3 – Redirection: `ls -l > out.txt`

The third program reproduces shell output redirection manually. The child closes descriptor 1, opens `out.txt`, and then executes `ls -l`. Because descriptor 1 is free, `open` reuses it.

redirect.c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls -l > out.txt   */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
```

```
        close(1);                                /* descriptor 1 is now free
    */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as always
    */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

The run confirmed that `open` returned descriptor 1, and the listing was written to the file instead of the terminal.

Output

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 88
-rw-r--r-- 1 findredhundred findredhundred     0 Oct 19 15:05 out.txt
-rwxrwxr-x 1 findredhundred findredhundred 16296 Oct 19 15:04 pipeline
```

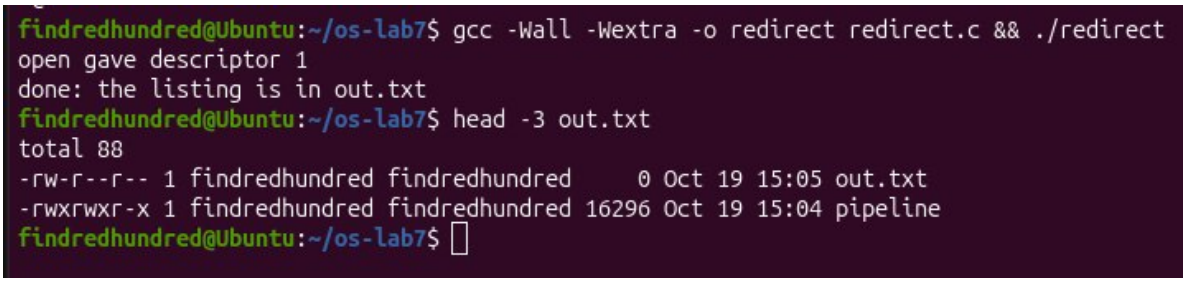


Figure 3.1 – Redirecting descriptor 1 to `out.txt`.

4.1. Observations

1. Which descriptor did `open` return, and why?

Open gave number 1, because we have closed file descriptor 1 which was stdout, and `open` opens the file in the smallest available file descriptor.

2. Why does the `ls` listing not appear on the screen?

Because `ls` writes to file descriptor 1 and it was changed to be `out.txt` prior to `exec`. From the point of view of `ls`, it is still simply writing to standard output.

3. Which descriptor would be closed to redirect errors instead?

Close descriptor 2, as its stderr.

5. Part 4 – A Pipe: `ls /etc | wc -l`

The last program builds a pipeline by hand using one pipe and two children. The first child turns its standard output into the writing end of the pipe, while the second child turns its standard input into the reading end.

pipeline.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command: ls /etc | wc -l */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                  /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                  /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
```

```
}
```

With all pipe ends closed correctly, my program and the shell pipeline printed the same result:

Output

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
240
$ ls /etc | wc -l
240
```

I then commented out the parent's `close(p[1])`. The program no longer terminated and I stopped it using Ctrl-C.



Figure 4.1 – The manual pipeline gives the same count as the shell, then hangs after leaving the write end open.

5.1. Observations

1. Did both versions give the same number?

Yes. Both `./pipeline` and `ls /etc | wc -l` printed 240.

2. Which child never ends after the change?

The `wc` child never ends because it still waits for input to be written as the input side of the pipe is still open.

3. What exactly is `wc` waiting for?

It is waiting for more input from the parent. More precisely, because the parent still has a write end of the pipe open, `wc` never receives end-of-file on its input and therefore keeps waiting for more data.

6. Conclusion

This laboratory made the process model behind normal shell commands much more visible. In the first part I saw how `fork`, `exec` and `wait` work together, including why failed `exec` calls return to the old code and can produce status 127. The zombie experiment showed why a finished child still needs to be collected by its parent. Redirection then showed that commands do not need to know they are writing into a file: changing descriptor 1 before `exec` is enough. Finally, the pipe experiment showed that a shell pipeline is just file descriptors connected between processes, and that leaving even one write end open is enough to keep `wc` waiting forever.